

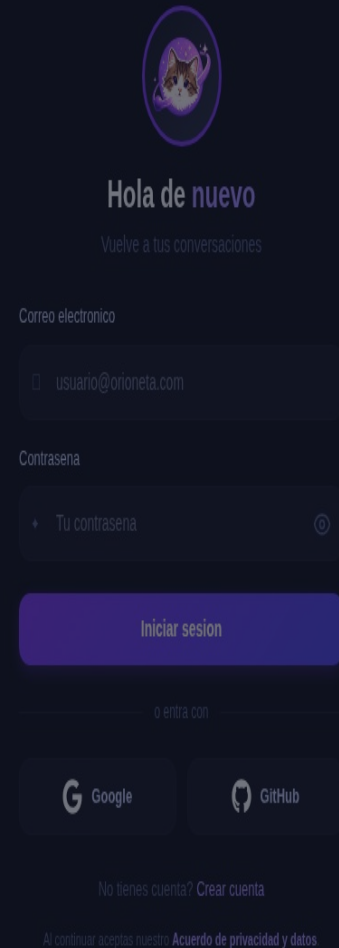
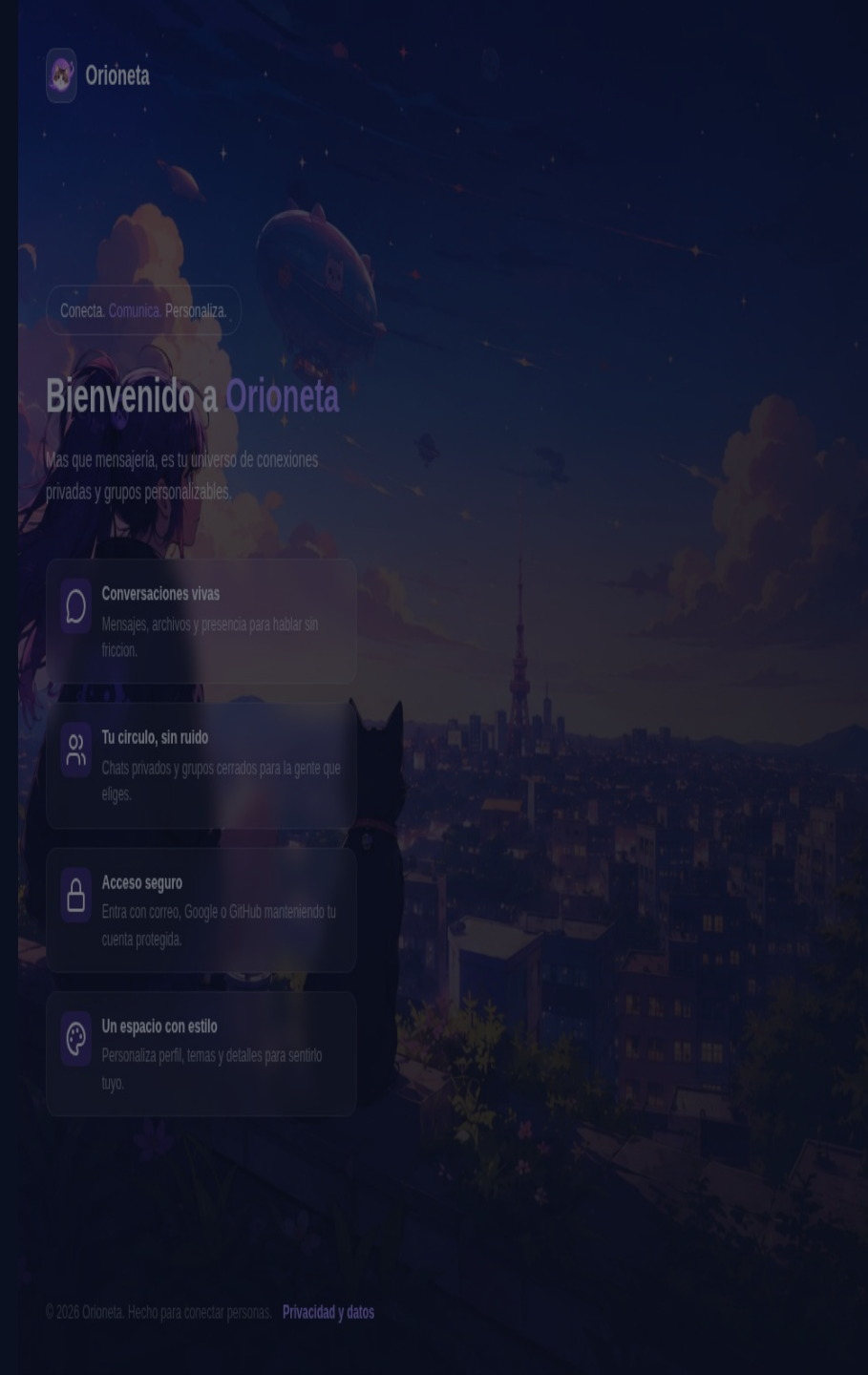
ORIONETA

CI/CD y despliegue en Amazon EKS

Evaluacion Final Transversal | ISY1101

Leandro Pedraza / Juan [completar apellido]

<https://orioneta.accesscam.org>



Objetivo y alcance

Del commit a una aplicación accesible por HTTPS

- Automatizar build, pruebas, imágenes y despliegue.
- Integrar frontend, 14 microservicios y datos relacionales.
- Ejecutar todos los componentes en contenedores.
- Orquestar producción con Amazon EKS.
- Evidenciar seguridad, logs, métricas y trazabilidad.

Resultado verificable

Frontend 200 OK, Gateway UP, OAuth disponible, WebSocket WSS y 14/14 Deployments listos.

Tecnologías

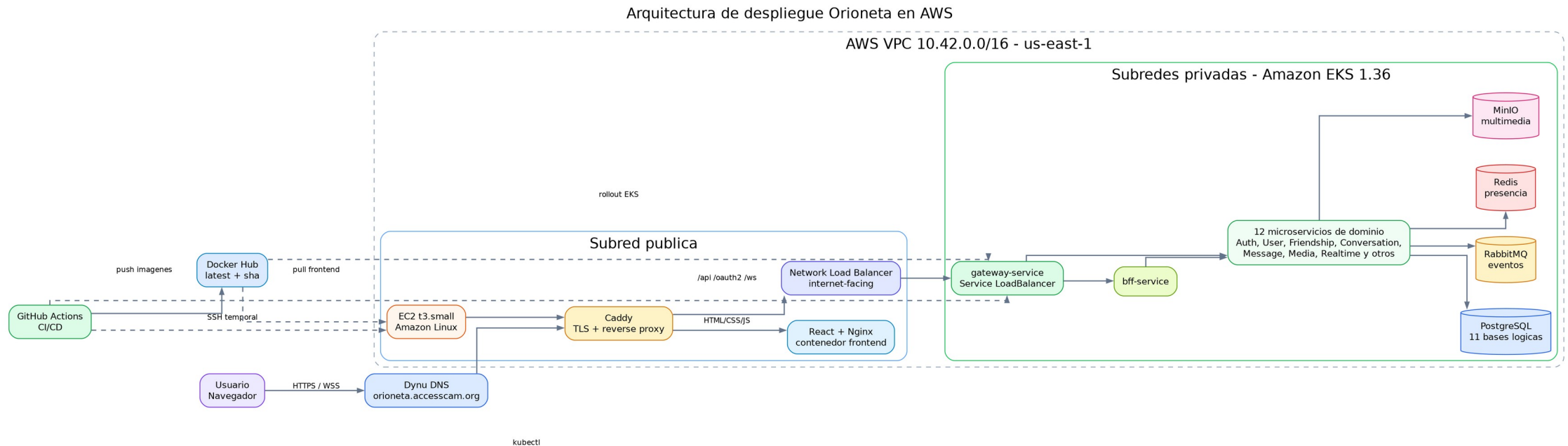
React 19, Vite 8, Java 25, Spring Boot, Maven, PostgreSQL, RabbitMQ, Redis, MinIO, Docker, Kubernetes y AWS.

Repositorios públicos

Backend: OrionTheProgrammer/Orioneta-Backend
Frontend: Panditax727/Orioneta-Frontend
Rama productiva: main

Arquitectura desplegada

Separacion entre acceso publico y servicios internos



Dynu + Caddy

EC2 frontend

NLB + Gateway

EKS privado

EBS + datos

Como se integra el sistema

Cada tecnologia resuelve un tipo de comunicacion

HTTPS same-origin

Caddy entrega React y reenvia /api, /oauth2 y /ws. El navegador utiliza un unico dominio y evita CORS innecesario.

REST + BFF

Gateway centraliza la entrada. El BFF compone datos y los servicios se resuelven por DNS interno de Kubernetes.

Eventos + realtime

RabbitMQ desacopla mensajes y notificaciones. WebSocket entrega eventos; Redis mantiene presencia.

PostgreSQL

Un motor compartido en el laboratorio con una base logica por servicio para evitar acceso cruzado a tablas.

MinIO

Los objetos multimedia se almacenan fuera de la base; media-service conserva metadatos y autorizacion.

JWT propio

Google y GitHub validan identidad, pero auth-service emite el token interno usado por Orioneta.

Contenedores y entorno local

Imágenes pequeñas, reproducibles y sin privilegios

Backend Java

Dos etapas
JRE 25 Alpine
Usuario orioneta
Solo JAR y puerto
.dockerignore reducido

Frontend React

Node 22 Alpine compila
Nginx Alpine sirve dist
Healthcheck /health
Sin dependencias de desarrollo

Docker Compose

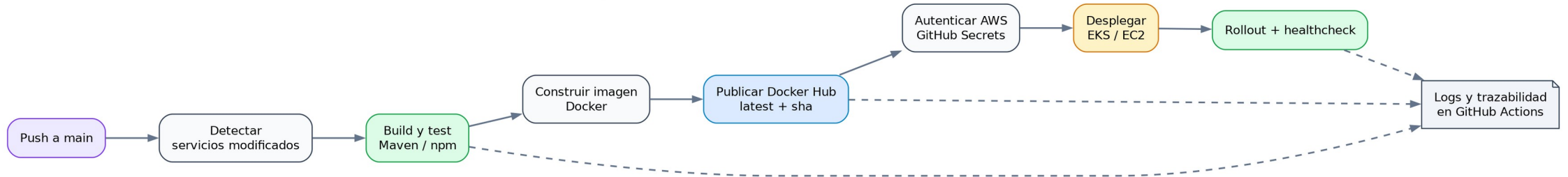
Infraestructura para IDE
Pila completa con perfiles
Frontend + Caddy + API
Volumenes persistentes
localhost:5173

Perfiles: messaging | realtime | customization | media | market | audit | observability

Pipeline CI/CD

Automatizado desde main y trazable por SHA

Pipeline CI/CD automatizado desde main



Backend

Deteccion de cambios -> Maven verify -> Docker Hub -> Kustomize -> rollout EKS.

Frontend

Build Vite -> Docker Hub -> SSH temporal -> EC2/Caddy -> healthcheck -> revocar acceso.

Tags: latest + sha | Secretos enmascarados | Concurrency evita despliegues superpuestos

Infraestructura AWS

Red segmentada y backend orquestado

VPC

10.42.0.0/16
2 zonas de disponibilidad
2 subredes publicas
2 subredes privadas

Frontend

EC2 t3.small
Amazon Linux
Caddy + Nginx
80/443 publicos

Backend

EKS 1.36
NLB internet-facing
14 Deployments
Services ClusterIP

Persistencia

PVC con EBS gp3 cifrado para PostgreSQL, RabbitMQ, Redis y MinIO.

Alta disponibilidad

Subredes en us-east-1a y us-east-1b. El NLB distribuye entrada al Gateway.

Costos del laboratorio

Una replica por servicio. La arquitectura permite HPA y mas replicas al crecer.

Seguridad y secretos

Capas de control desde GitHub hasta el contenedor

- HTTPS automatico con Caddy.
- JWT interno y Resource Server.
- GitHub Secrets -> Kubernetes Secret.
- Servicios de datos sin exposicion publica.
- SSH abierto solo para la IP efimera del runner.
- Imagenes Alpine y usuario Java sin root.
- Mejora: OIDC, Trivy, NetworkPolicy y Secrets Manager.

Principio de minimo privilegio

El pipeline solo necesita publicar imagenes, autenticarse en EKS y actualizar recursos. Los secretos nunca se escriben en Git.

Superficie publica

EC2: 80/443

EKS: Gateway por NLB

Interno: bases, brokers y microservicios

Credenciales academicas

AWS Academy usa sesiones temporales. En una cuenta permanente se reemplazan por GitHub OIDC.

Pruebas y observabilidad

Calidad antes del deploy y salud despues del rollout

68

pruebas backend
0 fallos / 0 errores

37,11%

cobertura de instrucciones
JaCoCo

14/14

Deployments disponibles
0 reinicios observados

UP

Gateway readiness
HTTPS y WSS activos

```
AWS CLOUDWATCH - EC2 FRONTEND
=====
Instancia: i-07cabd7bb6df6de0e
Tipo: t3.small
Ventana: ultimas 6 horas al 21-06-2026

CPU promedio minima:      0,16 %
CPU promedio maxima:      1,13 %
CPU maxima observada:      3,44 %
NetworkIn maximo:         11.240.537 bytes
NetworkOut maximo:         389.451 bytes
StatusCheckFailed:         0

Interpretacion:
La instancia se encuentra saludable y con holgura de CPU.
El pico de red coincide con la descarga de la nueva imagen.
```

```
AMAZON EKS - NAMESPACE ORIONETA
=====
Cluster: orioneta-eks
Version: 1.36
Estado: ACTIVE

DEPLOYMENTS                                READY
gateway-service                            1/1
bff-service                                1/1
auth-service                               1/1
user-service                               1/1
friendship-service                         1/1
conversation-service                       1/1
message-service                            1/1
notification-service                       1/1
customization-service                     1/1
media-service                              1/1
neta-market-service                       1/1
realtime-service                           1/1
moderation-service                        1/1
audit-service                              1/1

ESTADO ADICIONAL
PostgreSQL, RabbitMQ, Redis y MinIO: Running
Reinicios observados: 0
Gateway: Network Load Balancer activo
```

Evidencia de automatizacion

Dos repositorios, dos pipelines, una version productiva

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search or jump to...

Sign in

Sign up

OrionTheProgrammer / Orioneta-Backend

Notifications

Fork 0

Star 2

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

← Build, publish and deploy Orioneta

merge(eft): integrar entrega final DevOps #34

Sign in to view logs

Summary

All jobs

Detectar servicios modificados

Probar y publicar imagenes

Desplegar en Amazon EKS

Run details

Usage

Workflow file

Triggered via push 7 minutes ago

OrionTheProgrammer pushed · 0721e02 main

Success

Total duration 7m 10s

Artifacts -

dockerhub-images.yml

on: push

Detectar servicios modificados 5s

Probar y publicar image... 3m 52s

Desplegar en Amazon EKS 3m 2s

Backend: test + imagenes + EKS

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search or jump to...

Sign in

Sign up

Panditax727 / Orioneta-Frontend

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

← Build and deploy frontend

merge(eft): integrar documentacion y CI del frontend #3

Sign in to view logs

Summary

All jobs

build-and-push

deploy

Run details

Usage

Workflow file

Triggered via push 8 minutes ago

OrionTheProgrammer pushed · 76b2d9a main

Success

Total duration 54s

Artifacts 1

deploy-frontend.yml

on: push

build-and-push 31s

deploy 16s

Artifacts

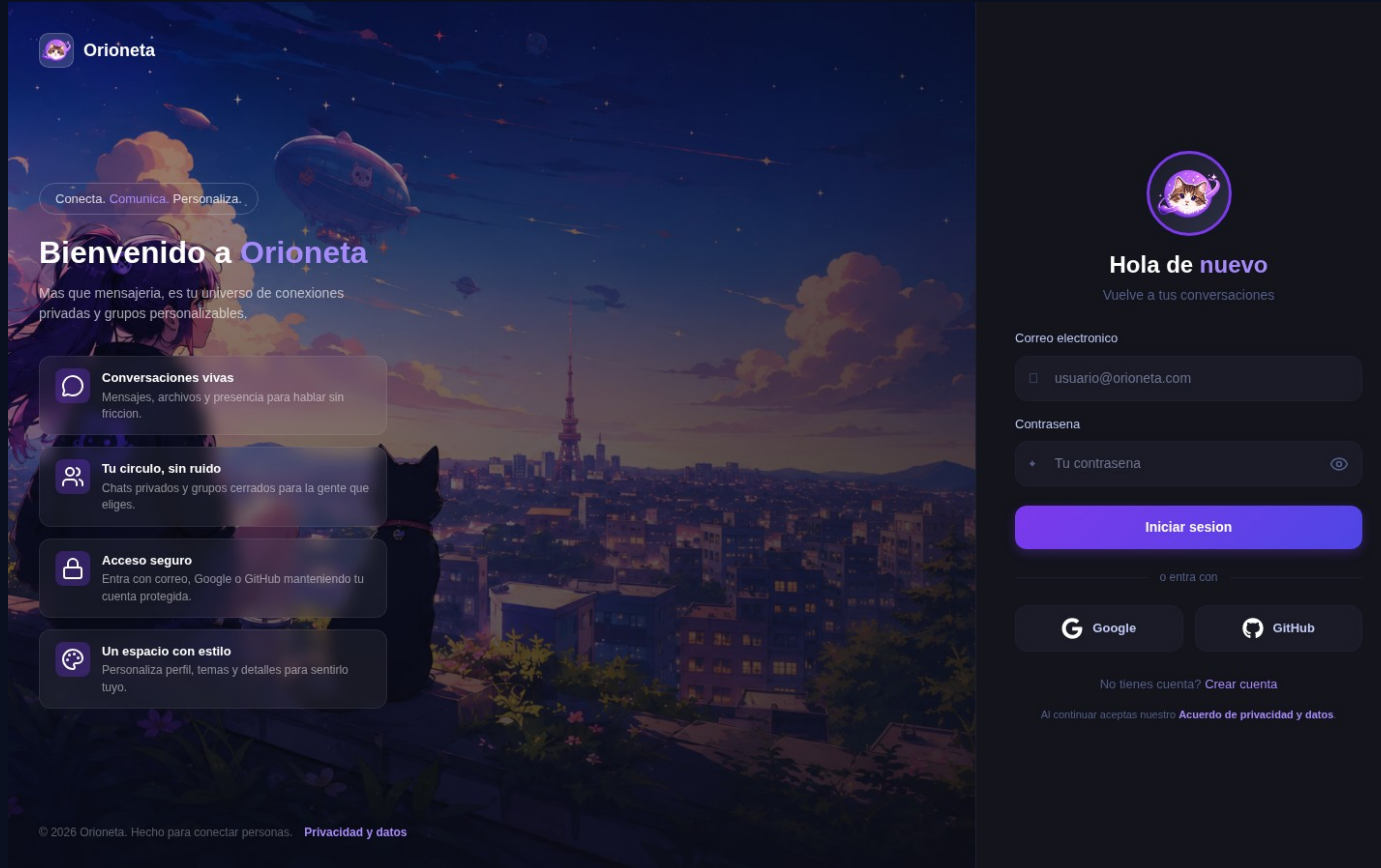
Produced during runtime

Name	Size	Digest
Panditax727-Orioneta-Frontend-97MXC6.dockerbuild	22.9 KB	sha256:98002acdba3c7e1cf4a6fe699fd2b2f5e67_

Frontend: imagen + EC2 + Caddy

Demostracion en vivo

Recorrido de 2 minutos



1 HTTPS y acceso

2 Login y chat

3 Actions exitosas

4 kubectl: pods y services

5 CloudWatch

Plan B: capturas y enlaces del informe

Conclusiones

Automatización reproducible, trazable y escalable

- Cada cambio validado produce un artefacto identificable.
- EKS reemplaza operación manual por estado declarativo.
- La red limita la exposición a los puntos necesarios.
- Actuator, Actions y CloudWatch entregan evidencia operativa.
- La solución está preparada para HPA y servicios administrados.

Proximos pasos

GitHub OIDC
Escaneo Trivy
Container Insights
HPA y pruebas de carga
RDS + S3

Preguntas

<https://orioneta.accesscam.org>